

```

board = ["1", "2", "3",
         "4", "5", "6",
         "7", "8", "9"]

players = {
    "X": "Player 1",
    "O": "Player 2"
}

winning_combinations = (
    (0, 1, 2),
    (3, 4, 5),
    (6, 7, 8),
    (0, 3, 6),
    (1, 4, 7),
    (2, 5, 8),
    (0, 4, 8),
    (2, 4, 6)
)

occupied = set()

def display_board():
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("---+---+---")
    print(board[3], "|", board[4], "|", board[5])
    print("---+---+---")
    print(board[6], "|", board[7], "|", board[8])
    print()

def check_winner(symbol):
    for combination in winning_combinations:
        if all(board[position] == symbol for position in combination):
            return True
    return False

def make_move(symbol):
    while True:
        choice = input(
            f"{players[symbol]} ({symbol}), choose a position (1-9): "
        )

        if not choice.isdigit():
            print("Please enter a valid number.")
            continue

        position = int(choice) - 1

        if position < 0 or position > 8:
            print("Please choose a position between 1 and 9.")
            continue

        if position in occupied:
            print("This position is already occupied.")
            continue

        board[position] = symbol
        occupied.add(position)
        break

print("Welcome to Tic-Tac-Toe!")

current_player = "X"

while True:
    display_board()

    make_move(current_player)

    if check_winner(current_player):
        display_board()
        print(players[current_player], "wins!")
        break

    if len(occupied) == 9:
        display_board()
        print("It's a draw!")
        break

    current_player = "O" if current_player == "X" else "X"

```

```
print("\nThanks for playing!")
```

```
1 | 2 | 3
---+---
4 | X | 6
---+---
7 | 8 | 9
```

Player 2 (O), choose a position (1-9): 6

```
1 | 2 | 3
---+---
4 | X | 0
---+---
7 | 8 | 9
```

Player 1 (X), choose a position (1-9): 1

```
X | 2 | 3
---+---
4 | X | 0
---+---
7 | 8 | 9
```

Player 2 (O), choose a position (1-9): 9

```
X | 2 | 3
---+---
4 | X | 0
---+---
7 | 8 | 0
```

Player 1 (X), choose a position (1-9): 3

```
X | 2 | X
---+---
4 | X | 0
---+---
7 | 8 | 0
```

Player 2 (O), choose a position (1-9): 7

```
X | 2 | X
---+---
4 | X | 0
---+---
0 | 8 | 0
```

Player 1 (X), choose a position (1-9): 2

```
X | X | X
---+---
4 | X | 0
---+---
0 | 8 | 0
```

Player 1 wins!

Thanks for playing!

```
# Vacuum Cleaner Agent Simulation with Performance Cost

def vacuum_world_with_cost():
    # 0 indicates CLEAN, 1 indicates DIRTY
    goal_state = {'A': '0', 'B': '0'}

    # User inputs for initial configuration
    location_input = input("Enter initial location of vacuum cleaner (A or B): ").strip().upper()
    status_input = input("Enter status of Location A (0 for Clean, 1 for Dirty): ").strip()
    status_input_2 = input("Enter status of Location B (0 for Clean, 1 for Dirty): ").strip()

    # Initialize the environment state
    environment_state = {'A': status_input, 'B': status_input_2}
    total_cost = 0

    print("\n--- Starting Vacuum Simulation ---")
    print(f"Initial State: {environment_state} | Vacuum Location: {location_input}\n")

    current_location = location_input

    # Run loop until environment matches the goal state (both rooms are clean)
    while environment_state != goal_state:
        if environment_state[current_location] == '1':
            # Location is dirty -> SUCK dirt
            print(f"[Location {current_location}] is DIRTY.")
```

```

environment_state[current_location] = '0'
total_cost += 2 # High energy cost for vacuuming/sucking
print(f"Action: SUCK | Environment updated: {environment_state} | Cost added: 2 (Total: {total_cost})")

# Check if the goal state is achieved mid-loop to avoid redundant movement
if environment_state == goal_state:
    break

# Move to the other room if current room is clean
if current_location == 'A':
    print(f"[Location A] is CLEAN. Moving Right...")
    current_location = 'B'
    total_cost += 1 # Cost for moving left/right
    print(f"Action: MOVE RIGHT | Current Location: {current_location} | Cost added: 1 (Total: {total_cost})")
else:
    print(f"[Location B] is CLEAN. Moving Left...")
    current_location = 'A'
    total_cost += 1 # Cost for moving left/right
    print(f"Action: MOVE LEFT | Current Location: {current_location} | Cost added: 1 (Total: {total_cost})")

print("\n--- Goal State Achieved! ---")
print(f"Final Environment State: {environment_state}")
print(f"Total Performance Cost: {total_cost}\n")

if __name__ == "__main__":
    vacuum_world_with_cost()

```

```

Enter initial location of vacuum cleaner (A or B): A
Enter status of Location A (0 for Clean, 1 for Dirty): 0
Enter status of Location B (0 for Clean, 1 for Dirty): 1

--- Starting Vacuum Simulation ---
Initial State: {'A': '0', 'B': '1'} | Vacuum Location: A

[Location A] is CLEAN. Moving Right...
Action: MOVE RIGHT | Current Location: B | Cost added: 1 (Total: 1)
[Location B] is DIRTY.
Action: SUCK | Environment updated: {'A': '0', 'B': '0'} | Cost added: 2 (Total: 3)

--- Goal State Achieved! ---
Final Environment State: {'A': '0', 'B': '0'}
Total Performance Cost: 3

```